KenKen Solver Project

Abstract

The KenKen Solver project is designed to solve KenKen puzzles using two main approaches: Backtracking and Cultural Algorithm. This project demonstrates the practical application of classical and evolutionary AI techniques to constraint-based puzzles. By implementing these algorithms in Python, the project explores problem-solving strategies for combinatorial and arithmetic challenges and analyzes their performance and efficiency.
here is accessible link :
https://drive.google.com/drive/folders/1gwuCVTMbIVzCo4MBd6vtFKS0b81YoUMi?usp=sharing

Project Idea

KenKen is a grid-based puzzle similar to Sudoku but with additional arithmetic constraints grouped in cages. Each cage specifies a target value and an operation (addition, multiplication, subtraction, or division) that must be satisfied by the numbers in the cells it contains. The goal is to fill the grid with numbers from 1 to the size of the grid without repeating numbers in any row or column and while satisfying all cage constraints.

Team members:

Ahmed Mohamed Abdel Gawad	20230042	IS
Ahmed Mohamed Ahmed Hussien	20230036	IS
Ahmed Mohamed Metwally	20230047	IS
Safwa Saad Ali Mohamed	20230296	IT
Hanin Mohamed Tawfik	20230184	IS
Shahd Magdy Mohamed Amer	20230286	IT

The project implements a solver that can apply two distinct methods:

1. Backtracking Algorithm – systematically explores possibilities for each empty cell while respecting all constraints.

2. Cultural Algorithm – an evolutionary approach that evolves a population of candidate solutions, selecting and recombining them over multiple generations to find valid solutions.

The project highlights the effectiveness of different algorithmic approaches in solving constraint satisfaction problems, reflecting practical AI and heuristic techniques.

Methodology

1. Backtracking Algorithm

The Backtracking approach in this project is designed to guarantee a correct solution by systematically exploring possible number placements:

- Sequential Exploration: The algorithm examines empty cells one by one and tries all possible numbers.

- Constraint Checking: It ensures that the placement of a number respects row uniqueness, column uniqueness, and cage arithmetic constraints.

- Backtracking: If a conflict arises, the algorithm returns to the previous cell to try alternative numbers.

- Heuristics: The solver uses the Minimum Remaining Values (MRV) heuristic to prioritize cells with fewer valid options and forward checking to eliminate impossible values early, improving efficiency.

- Visualization: The solver optionally visualizes the assignment of numbers to cells to observe the solving process.

2. Cultural Algorithm

The Cultural Algorithm is an evolutionary algorithm that solves the KenKen puzzle using population-based optimization **:**

- Population Initialization: A population of candidate solutions is generated with constraints considered to produce feasible initial solutions.

- Fitness Evaluation: Each candidate solution is evaluated based on the number of constraint violations, including row and column uniqueness and cage arithmetic operations.

- Elite Selection: The best-performing solutions (elites) are selected to influence the next generation.
- Crossover and Mutation: Selected candidates are recombined and mutated to create new solutions, maintaining diversity and improving solution quality.

- Stagnation Handling: If no improvement occurs over several generations, the algorithm restarts with a new population to avoid stagnation and enhance the likelihood of finding a solution.

- Visualization: The algorithm can show intermediate best solutions for educational or debugging purposes.

- Grid Representation: Candidate solutions are represented as chromosomes converted to grid format, ensuring consistent mapping to the puzzle.

Implementation Steps

1. Load Puzzle: The solver reads the puzzle's size, cage definitions, and operations from a file or user input.

2. Algorithm Selection: The user chooses the Backtracking method or Cultural Algorithm.

3. Solve Puzzle:

    - Backtracking: Recursively fills cells while checking constraints and backtracking when necessary.

    - Cultural Algorithm: Evolves a population over multiple generations, applying fitness evaluation, elite selection, crossover, and mutation.

4. Display Solution: The final solution is shown via the interface, which could be a GUI or console output.

5. Performance Monitoring: Execution metrics such as number of generations, retries, or solving time are optionally tracked to evaluate efficiency.

Detailed Discussion

Backtracking

The Backtracking solver provides a deterministic method that ensures a correct solution. The use of MRV and forward checking significantly reduces the search space. By sequentially exploring valid numbers for each cell and backtracking when necessary, the solver guarantees puzzle completion. This approach directly mirrors the logic in the Python implementation, with careful consideration of row, column, and cage constraints.

Cultural Algorithm

The Cultural Algorithm offers a stochastic, population-based solution method. By initializing a population with feasible candidates and evaluating their fitness according to puzzle rules, the algorithm iteratively improves the solution quality. Elite selection, crossover, and mutation maintain genetic diversity while promoting better solutions. Stagnation detection and retries prevent the algorithm from getting stuck in local optima. This approach reflects the implementation structure in the Python code and demonstrates evolutionary problem-solving techniques.

Comparative Analysis

Feature	Backtracking	Cultural Algorithm
Deterministic	Yes, guarantees a solution if one exists	No, stochastic; may require retries if stagnation occurs
Execution Flow	Recursive exploration, cell-by-cell	Population initialization, evaluation, selection, crossover, mutation over generations
Constraint Handling	Directly checks rows, columns, and cage operations	Evaluates fitness based on row, column uniqueness and cage constraints
Efficiency Optimizations	MRV heuristic, forward checking	Elite selection, mutation, crossover, stagnation detection
Solution Guarantee	Always finds correct solution	May find approximate solution or best solution after retries
Visualization Support	Optional per-cell assignment display	Optional display of the final solution and if solution not found display the best solution
Performance Considerations	Slower on larger grids due to exhaustive search |	Faster on larger grids; may require more memory for population management
Adaptability	Focused on a single puzzle at a time	Can explore multiple solution paths simultaneously; adaptable to varied constraints

Results and Analysis

Both algorithms successfully solve KenKen puzzles, with performance varying by puzzle size and complexity.

Backtracking excels in guaranteed correctness but may consume more time for larger grids.

Cultural Algorithm is faster in many scenarios but may require additional attempts to reach a perfect solution.

Observations indicate that evolutionary methods can complement classical algorithms, offering practical insights for AI-based problem-solving in constraint satisfaction tasks.

Discussion & Analysis of Results

In the Cultural Algorithm implemented:

1. **Parents' Selection Approach**

   The algorithm selects two parents randomly from the elite population for crossover (p1, p2 = random.sample(elites, 2)).

   This selection ensures diversity while still favoring the best-performing candidates.

2. **Crossover Approach**

   Row-based crossover preserves row uniqueness (child.extend(random.choice([row_from_p1, row_from_p2]))).

   This ensures that offspring are valid candidates while combining genetic information from two parents.

3. **Mutation Approach**

   Mutation swaps two numbers in a row with a certain rate (mutate(ind, rate=0.1)).

   Helps maintain diversity and prevent premature convergence to suboptimal solutions.

4. **Population Size**

   The population is initialized with pop_size=700.

   Larger populations increase diversity but also computational cost; smaller populations are faster but may stagnate early.

5. **Belief-Space Parameters**

   Not explicitly implemented as a separate belief-space, but elite selection acts as a cultural memory guiding the evolution of the population.

   Elites influence offspring generation, effectively embedding "beliefs" about high-fitness solutions.

6. **Survivors' Selection & Elitism**

   Top elites_count=30 candidates are preserved each generation.

   This ensures the best solutions survive and guide the next generation, improving convergence reliability.

7. **Effect on Results**

   Proper balance between crossover, mutation, population size, and elitism is crucial.

   High elitism or small population may reduce diversity, causing stagnation (handled by restart after stagnation detection).

   Mutation rate affects exploration: too low slows improvement, too high disrupts good candidates.

Conclusion

The KenKen Solver project demonstrates two effective approaches for solving constraint-based puzzles. The Backtracking method provides reliability and correctness, while the Cultural Algorithm illustrates evolutionary problem-solving techniques. By comparing the algorithms based on implementation and performance, this project offers insights into the strengths and trade-offs of classical and AI-based methods for combinatorial challenges.

References / Appendix

- Reynolds, Robert. (1994). “An Introduction to Cultural Algorithms.”
- D. B. Fogel, “An Introduction to Simulated Evolutionary Optimization”
- DR/Amr Ghoneim slides and videos
- KenKen puzzle rules and sample puzzles.
- Execution observations, including solving time, number of iterations, and retry attempts.
- Optional GUI screenshots showing completed puzzles.